Appendix A: Source Code

server.js

```
const express = require("express");
const path = require("node:path");

const app = express();
const port = process.env.PORT || 3000;
const host = process.env.HOST || "127.0.0.1";
const apiKey = process.env.OPENWEATHER_API_KEY;
const mapboxToken = process.env.MAPBOX_ACCESS_TOKEN || "";

// send the public token so the map can load
app.get("/api/config", (req, res) => {
    res.json({ mapboxToken });
});

// look up weather for a typed city
app.get("/api/weather", async (req, res) => {
    const city = typeof req.query.city === "string" ? req.query.city.trim() : "";

    if (!city || city.length > 100) {
        return res.status(400).json({
            error: "Enter a city name, optionally followed by a country code."
        });
    }

    if (!apiKey) {
        return res.status(500).json({
            error: "The OpenWeather API key is missing. Add it to the .env file and restart
the server."
        });
    }

    try {
        return res.json(await getWeatherForCity(city));
    } catch (error) {
        return res.status(error.status || 502).json({ error: error.message });
    }
});

// look up weather for a clicked point
app.get("/api/weather-at", async (req, res) => {
    const latitude = typeof req.query.lat === "string" ? req.query.lat.trim() : "";
    const longitude = typeof req.query.lon === "string" ? req.query.lon.trim() : "";
    const lat = latitude ? Number(latitude) : NaN;
    const lon = longitude ? Number(longitude) : NaN;

    if (
        !Number.isFinite(lat) || !Number.isFinite(lon) ||
        lat < -90 || lat > 90 || lon < -180 || lon > 180
    ) {
        return res.status(400).json({ error: "Choose a valid point on the map." });
    }

    if (!apiKey) {
        return res.status(500).json({
            error: "The OpenWeather API key is missing. Add it to the .env file and restart
the server."
        });
    }

    try {
        const location = await getLocationForCoordinates(lat, lon);
```

```
        return res.json(await getWeatherForCoordinates(lat, lon, location));
    } catch (error) {
        return res.status(error.status || 502).json({ error: error.message });
    }
});

// get weather map tiles without exposing the openweather key
app.get("/api/weather-map/:layer/:z/:x/:y.png", async (req, res) => {
    const layer = req.params.layer;
    const zoom = Number(req.params.z);
    const tileX = Number(req.params.x);
    const tileY = Number(req.params.y);
    const tilesAtZoom = 2 ** zoom;

    if (
        !["precipitation_new", "temp_new"].includes(layer) ||
        ![zoom, tileX, tileY].every(Number.isInteger) ||
        zoom < 0 || zoom > 18 ||
        tileX < 0 || tileY < 0 ||
        tileX >= tilesAtZoom || tileY >= tilesAtZoom
    ) {
        return res.sendStatus(400);
    }

    if (!apiKey) {
        return res.sendStatus(500);
    }

    const tileUrl = new URL(
        `https://tile.openweathermap.org/map/${layer}/${zoom}/${tileX}/${tileY}.png`
    );
    tileUrl.search = new URLSearchParams({ appid: apiKey });

    try {
        const tileResponse = await fetch(tileUrl);
        if (!tileResponse.ok) {
            return res.sendStatus(tileResponse.status);
        }

        // send the tile image back to the map
        const image = Buffer.from(await tileResponse.arrayBuffer());
        return res.type("png").send(image);
    } catch {
        return res.sendStatus(502);
    }
});

app.use(express.static(path.join(__dirname, "public")));

// find the city coordinates first
async function getWeatherForCity(city) {
    // the weather requests need coordinates
    const locationUrl = new URL("https://api.openweathermap.org/geo/1.0/direct");
    locationUrl.search = new URLSearchParams({ q: city, limit: "1", appid: apiKey });
    const locations = await getOpenWeatherJson(locationUrl);

    if (!locations.length) {
        const error = new Error("City not found. Try adding its country code.");
        error.status = 404;
        throw error;
    }

    const location = locations[0];
    return getWeatherForCoordinates(location.lat, location.lon, location);
}
```

```
// city searches and map pins use the same weather lookup
async function getWeatherForCoordinates(lat, lon, location) {
    const currentUrl = new URL("https://api.openweathermap.org/data/2.5/weather");
    currentUrl.search = new URLSearchParams({
        lat: String(lat),
        lon: String(lon),
        units: "metric",
        appid: apiKey
    });
    const forecastUrl = new URL("https://api.openweathermap.org/data/2.5/forecast");
    forecastUrl.search = new URLSearchParams({
        lat: String(lat),
        lon: String(lon),
        units: "metric",
        appid: apiKey
    });
    // get current weather and forecast at the same time
    const [current, forecast] = await Promise.all([
        getOpenWeatherJson(currentUrl),
        getOpenWeatherJson(forecastUrl)
    ]);
    const timezoneOffset = forecast.city.timezone || 0;
    // keep today separate from the next three days
    const dailySummaries = makeDailySummaries(
        forecast.list,
        timezoneOffset
    );
    const today = dailySummaries[0];
    const days = dailySummaries.slice(1, 4);

    if (!today || !days.length) {
        const error = new Error("OpenWeather did not return enough forecast data.");
        error.status = 502;
        throw error;
    }

    // the forecast still works if air quality fails
    let airQuality;
    try {
        const airUrl = new URL("https://api.openweathermap.org/data/2.5/air_pollution");
        airUrl.search = new URLSearchParams({
            lat: String(lat),
            lon: String(lon),
            appid: apiKey
        });
        const airData = await getOpenWeatherJson(airUrl);
        airQuality = makeAirQualitySummary(airData.list[0]);
    } catch {
        airQuality = {
            available: false,
            message: "Air-quality data is unavailable."
        };
    }

    return {
        city: {
            name: location.name,
            state: location.state || "",
            country: location.country || ""
        },
        coordinates: {
            lat,
            lon
        },
```

```
        currentWeather: {
            temperature: current.main.temp,
            conditionDescription: current.weather[0].description,
            icon: current.weather[0].icon
        },
        mapSummary: makeMapSummary(forecast.list, timezoneOffset),
        today,
        days,
        packing: makePackingAdvice(days),
        airQuality,
        hasThreeDays: days.length === 3
    };
}

// find the next likely rain time for the map
function makeMapSummary(readings, timezoneOffset) {
    const now = Math.floor(Date.now() / 1000);
    const nextThreeDays = (readings || []).filter(
        (reading) => reading.dt >= now && reading.dt <= now + 3 * 24 * 60 * 60
    );
    // use 50% as the cutoff for likely rain
    const nextRain = nextThreeDays.find((reading) => (reading.pop || 0) >= 0.5);

    return {
        timezoneOffset,
        nextRain: nextRain
            ? {
                time: nextRain.dt,
                chance: Math.round((nextRain.pop || 0) * 100)
            }
            : null
    };
}

// get a place name for a map pin
async function getLocationForCoordinates(lat, lon) {
    try {
        const locationUrl = new URL("https://api.openweathermap.org/geo/1.0/reverse");
        locationUrl.search = new URLSearchParams({
            lat: String(lat),
            lon: String(lon),
            limit: "1",
            appid: apiKey
        });
        const locations = await getOpenWeatherJson(locationUrl);
        if (locations.length) return locations[0];
    } catch {
        // the coordinates still work if no place name is found
    }

    return { name: "Pinned location", state: "", country: "" };
}

async function getOpenWeatherJson(url) {
    let response;
    try {
        response = await fetch(url);
    } catch {
        const networkError = new Error("Could not connect to OpenWeather.");
        networkError.status = 502;
        throw networkError;
    }

    // read the response from openweather
    const data = await response.json();
```

```
    if (!response.ok) {
        const error = new Error(
            "OpenWeather request failed with status " + response.status + "."
        );
        error.status = response.status;
        throw error;
    }
    return data;
}

// group the three-hour readings by local date
function makeDailySummaries(readings, timezoneOffset) {
    const daysByDate = new Map();

    for (const reading of readings || []) {
        const date = new Date((reading.dt + timezoneOffset) * 1000)
            .toISOString()
            .slice(0, 10);
        if (!daysByDate.has(date)) daysByDate.set(date, []);
        daysByDate.get(date).push(reading);
    }

    return Array.from(daysByDate, ([date, dayReadings]) => {
        const middleReading = dayReadings[Math.floor(dayReadings.length / 2)];
        const condition = middleReading.weather[0];
        const temperatures = dayReadings.map((reading) => reading.main.temp);
        const lows = dayReadings.map((reading) => reading.main.temp_min ??
reading.main.temp);
        const highs = dayReadings.map((reading) => reading.main.temp_max ??
reading.main.temp);
        const average = (values) =>
            values.reduce((sum, value) => sum + value, 0) / values.length;
        const rainfall = dayReadings.reduce(
            (total, reading) => total + (reading.rain?.["3h"] || 0),
            0
        );
        const rainExpected = dayReadings.some((reading) => {
            const weather = reading.weather[0].main;
            return (
                (reading.rain?.["3h"] || 0) > 0 ||
                ["Rain", "Drizzle", "Thunderstorm"].includes(weather)
            );
        });
        const averageTemperature = average(temperatures);

        return {
            date,
            icon: condition.icon,
            conditionDescription: condition.description,
            minTemperature: Math.min(...lows),
            maxTemperature: Math.max(...highs),
            averageTemperature,
            temperatureCategory: getTemperatureCategory(averageTemperature),
            averageWindSpeed: average(dayReadings.map((reading) => reading.wind.speed)),
            rainfall,
            rainExpected
        };
    });
}

function getTemperatureCategory(temperature) {
    if (temperature < 8) return "Cold";
    if (temperature <= 24) return "Mild";
    return "Hot";
}
```

```
function makePackingAdvice(days) {
    const categories = [...new Set(days.map((day) => day.temperatureCategory))];
    let temperatureAdvice;

    if (categories.length > 1) {
        temperatureAdvice =
            "Pack layers. The daily average moves between " + categories.join(" and ") +
".";
    } else if (categories[0] === "Cold") {
        temperatureAdvice = "Pack a warm coat. The daily average is below 8°C.";
    } else if (categories[0] === "Hot") {
        temperatureAdvice = "Pack for warm weather. The daily average is above 24°C.";
    } else {
        temperatureAdvice = "A light layer should be enough for mild temperatures.";
    }

    const umbrellaNeeded = days.some((day) => day.rainExpected);
    return {
        umbrellaNeeded,
        umbrellaAdvice: umbrellaNeeded
            ? "Rain is forecast. Bring an umbrella."
            : "No rain is forecast in these three days.",
        temperatureAdvice
    };
}

function makeAirQualitySummary(reading) {
    const aqiLabels = ["", "Good", "Fair", "Moderate", "Poor", "Very poor"];
    const pollutants = [
        {
            key: "so2",
            name: "Sulphur dioxide (SO₂)",
            limits: [20, 80, 250, 350],
            healthRisk: "Can irritate airways and may worsen asthma symptoms."
        },
        {
            key: "no2",
            name: "Nitrogen dioxide (NO₂)",
            limits: [40, 70, 150, 200],
            healthRisk: "Can irritate airways and worsen respiratory symptoms."
        },
        {
            key: "pm10",
            name: "Particulate matter (PM₁₀)",
            limits: [20, 50, 100, 200],
            healthRisk: "Can affect the lungs and aggravate heart or lung conditions."
        },
        {
            key: "pm2_5",
            name: "Fine particulate matter (PM₂.₅)",
            limits: [10, 25, 50, 75],
            healthRisk: "Fine particles can enter the lungs and aggravate heart or lung
conditions."
        },
        {
            key: "o3",
            name: "Ozone (O₃)",
            limits: [60, 100, 140, 180],
            healthRisk: "Can irritate the lungs and make breathing harder."
        },
        {
            key: "co",
            name: "Carbon monoxide (CO)",
            limits: [4400, 9400, 12400, 15400],
```

```
            healthRisk: "At high exposure, it can reduce the oxygen carried through the
body."
        }
    ];

    const warnings = [];
    for (const pollutant of pollutants) {
        const value = reading.components[pollutant.key];
        if (!Number.isFinite(value) || value < pollutant.limits[0]) continue;

        const limitIndex = pollutant.limits.findIndex((limit) => value < limit);
        const categories = ["Fair", "Moderate", "Poor", "Very poor"];
        warnings.push({
            name: pollutant.name,
            value,
            category: limitIndex === -1 ? "Very poor" : categories[Math.max(0, limitIndex -
1)],
            healthRisk: pollutant.healthRisk
        });
    }

    const aqi = reading.main.aqi;
    return {
        available: true,
        aqi,
        aqiLabel: aqiLabels[aqi] || "Unknown",
        warnings
    };
}

app.listen(port, host, () => {
    console.log("Weather planner running at http://localhost:" + port);
});
```

public/app.js

```
let weatherMap = null;
let weatherMarker = null;

const app = Vue.createApp({
    data() {
        return {
            city: "",
            weatherData: null,
            error: "",
            loading: false,
            pinRequestInProgress: false,
            pinLoading: false,
            mapLoading: false,
            mapError: "",
            mapLayers: {
                precipitation: true,
                temperature: true
            }
        };
    },
    computed: {
        cityLabel() {
            return this.weatherData ? this.formatCityName(this.weatherData.city) : "";
        }
    },
    methods: {
        async searchWeather() {
            const cityName = this.city.trim();
            if (!cityName || this.loading || this.pinRequestInProgress) {
                return;
```

```
            }

            this.removeMap();
            this.loading = true;
            this.error = "";
            this.weatherData = null;
            this.mapLoading = false;
            this.mapError = "";

            try {
                // ask the server for this city's weather
                const response = await fetch("/api/weather?city=" +
encodeURIComponent(cityName));
                this.weatherData = await this.readJsonResponse(response);
            } catch (requestError) {
                this.error = requestError.message;
            } finally {
                this.loading = false;
                if (this.weatherData) {
                    await this.$nextTick();
                    await this.showWeatherMap();
                }
            }
        },
        async searchAtPoint(lat, lon) {
            if (this.loading || this.pinRequestInProgress) return;

            this.pinRequestInProgress = true;
            this.mapError = "";
            let loadingStartedAt = 0;
            const loadingTimer = window.setTimeout(() => {
                this.pinLoading = true;
                loadingStartedAt = Date.now();
            }, 180);

            try {
                const query = new URLSearchParams({
                    lat: String(lat),
                    lon: String(lon)
                });
                const response = await fetch("/api/weather-at?" + query);
                this.weatherData = await this.readJsonResponse(response);

                if (weatherMarker) {
                    weatherMarker.setLngLat([lon, lat]);
                }
            } catch (requestError) {
                this.mapError = "Could not load a forecast for that point. " +
requestError.message;
            } finally {
                window.clearTimeout(loadingTimer);
                if (loadingStartedAt) {
                    const remainingTime = 250 - (Date.now() - loadingStartedAt);
                    if (remainingTime > 0) {
                        await new Promise((resolve) => window.setTimeout(resolve,
remainingTime));
                    }
                }
                this.pinLoading = false;
                this.pinRequestInProgress = false;
            }
        },
        formatCityName(city) {
            return [city.name, city.state, city.country].filter(Boolean).join(", ");
        },
```

```
        async readJsonResponse(response) {
            // make sure the server sent json
            const contentType = response.headers.get("content-type") || "";
            if (!contentType.includes("application/json")) {
                throw new Error(
                    "The server returned a web page instead of JSON. Start or restart the
app with `npm start` from the project folder, then reload this page."
                );
            }
            const responseData = await response.json();
            if (!response.ok) {
                throw new Error(responseData.error || "The weather request failed. Please
try again.");
            }
            return responseData;
        },
        removeMap() {
            if (weatherMap) {
                weatherMap.remove();
                weatherMap = null;
                weatherMarker = null;
            }
        },
        setMapLayerVisibility(layerId, isVisible) {
            if (weatherMap && weatherMap.getLayer(layerId)) {
                weatherMap.setLayoutProperty(
                    layerId,
                    "visibility",
                    isVisible ? "visible" : "none"
                );
            }
        },
        async showWeatherMap() {
            if (!window.mapboxgl) {
                this.mapError = "The map library could not be loaded.";
                return;
            }

            const { lat, lon } = this.weatherData.coordinates;
            const mapboxgl = window.mapboxgl;
            this.mapLoading = true;
            this.mapError = "";

            try {
                const response = await fetch("/api/config");
                const config = await this.readJsonResponse(response);
                if (!config.mapboxToken) {
                    throw new Error("The Mapbox token is missing. Add it to .env and
restart the server.");
                }
                mapboxgl.accessToken = config.mapboxToken;
            } catch (error) {
                this.mapLoading = false;
                this.mapError = error.message;
                return;
            }

            const map = new mapboxgl.Map({
                container: "weather-map",
                style: "mapbox://styles/mapbox/standard",
                center: [lon, lat],
                zoom: 6.5
            });
            weatherMap = map;
```

```
            map.addControl(new mapboxgl.NavigationControl({ showCompass: false }),
"top-right");
            weatherMarker = new mapboxgl.Marker({ color: "#326b84" })
                .setLngLat([lon, lat])
                .addTo(map);

            map.on("click", (event) => {
                this.searchAtPoint(event.lngLat.lat, event.lngLat.lng);
            });

            map.once("load", () => {
                if (weatherMap !== map) return;

                map.addSource("temperature", {
                    type: "raster",
                    tiles: ["/api/weather-map/temp_new/{z}/{x}/{y}.png"],
                    tileSize: 256,
                    maxzoom: 18,
                    attribution: "© OpenWeather"
                });
                map.addSource("precipitation", {
                    type: "raster",
                    tiles: ["/api/weather-map/precipitation_new/{z}/{x}/{y}.png"],
                    tileSize: 256,
                    maxzoom: 18,
                    attribution: "© OpenWeather"
                });

                map.addLayer({
                    id: "temperature-layer",
                    type: "raster",
                    source: "temperature",
                    slot: "middle",
                    layout: {
                        visibility: this.mapLayers.temperature ? "visible" : "none"
                    },
                    paint: {
                        "raster-opacity": 0.4,
                        "raster-contrast": 0.3,
                        "raster-saturation": 0.2,
                        "raster-fade-duration": 0
                    }
                });
                map.addLayer({
                    id: "precipitation-layer",
                    type: "raster",
                    source: "precipitation",
                    slot: "middle",
                    layout: {
                        visibility: this.mapLayers.precipitation ? "visible" : "none"
                    },
                    paint: {
                        "raster-opacity": 0.78,
                        "raster-contrast": 0.3,
                        "raster-saturation": 0.2,
                        "raster-fade-duration": 0
                    }
                });
                this.mapLoading = false;
            });

            map.on("error", (event) => {
                if (weatherMap !== map) return;
                if (
                    !map.isStyleLoaded() ||
```

```
                event.sourceId === "precipitation" ||
                event.sourceId === "temperature"
            ) {
                this.mapLoading = false;
                this.mapError = "The map or a weather layer could not be loaded.";
            }
        });
    },
    formatDate(dateString, short = false) {
        const date = new Date(dateString + "T12:00:00Z");
        return new Intl.DateTimeFormat("en-IE", {
            weekday: short ? "short" : "long",
            day: "numeric",
            month: "short",
            timeZone: "UTC"
        }).format(date);
    },
    weatherIcon(iconCode) {
        // match the weather code to one of our icons
        const code = String(iconCode || "");
        const group = code.slice(0, 2);
        const night = code.endsWith("n");

        if (group === "01") return night ? "moon" : "sun";
        if (group === "02" || group === "03") {
            return night ? "partly-cloudy-night" : "partly-cloudy-day";
        }
        if (group === "04") return "cloud";
        if (group === "09" || group === "10") return "rain";
        if (group === "11") return "storm";
        if (group === "13") return "snow";
        if (group === "50") return "mist";
        return "cloud";
    },
    formatNumber(value) {
        if (!Number.isFinite(value)) {
            return "—";
        }
        const cleanValue = Math.abs(value) < 0.05 ? 0 : value;
        return new Intl.NumberFormat("en-IE", {
            minimumFractionDigits: 0,
            maximumFractionDigits: 1
        }).format(cleanValue);
    },
    formatTemperature(value) {
        return this.formatNumber(value) + "°C";
    },
    formatRainfall(value) {
        if (!Number.isFinite(value)) {
            return "—";
        }
        if (value === 0) {
            return "0";
        }
        if (value < 0.05) {
            return "<0.1";
        }
        return this.formatNumber(value);
    },
    formatForecastTime(timestamp, timezoneOffset) {
        const localDate = new Date((timestamp + timezoneOffset) * 1000);
        const day = new Intl.DateTimeFormat("en-IE", {
            weekday: "short",
            timeZone: "UTC"
        }).format(localDate);
```

```
            const time = new Intl.DateTimeFormat("en-IE", {
                hour: "numeric",
                hourCycle: "h12",
                timeZone: "UTC"
            }).format(localDate);
            return day + " at " + time;
        }
    }
});

app.mount("#app");
```

public/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="theme-color" content="#e8f5fb">
    <meta
      name="description"
      content="A simple three-day weather forecast, packing advice and current air-quality
information.">
    <title>Weather forecast</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link

href="https://fonts.googleapis.com/css2?family=Karla:wght@400;500;600;700&display=swap"
      rel="stylesheet">
    <link rel="stylesheet" href="style.css">
    <link href="https://api.mapbox.com/mapbox-gl-js/v3.32.0/mapbox-gl.css"
rel="stylesheet">
    <script src="https://unpkg.com/vue@3/dist/vue.global.prod.js" defer></script>
    <script src="https://api.mapbox.com/mapbox-gl-js/v3.32.0/mapbox-gl.js" defer></script>
    <script src="app.js" defer></script>
  </head>
  <body>
    <div id="app">
      <div class="weather-shell">
        <aside class="side-panel">
          <form class="search-form" @submit.prevent="searchWeather" :aria-busy="loading ||
pinRequestInProgress">
            <label for="city">Search for a city</label>
            <input
              id="city"
              v-model.trim="city"
              type="search"
              name="city"
              maxlength="100"
              autocomplete="off"
              placeholder="Dublin, IE"
              aria-describedby="city-hint"
              required>
            <button type="submit" :disabled="loading || pinRequestInProgress ||
!city.trim()">Search</button>
            <p id="city-hint" class="search-hint">
              Add a country code if the city name is shared.
            </p>
          </form>

          <section v-if="weatherData" class="today-summary" aria-labelledby="city-name">
            <h2 id="city-name" class="city-name">{{ cityLabel }}</h2>
            <p class="day-label">{{ formatDate(weatherData.today.date) }}</p>
```

```
            <div class="current-condition">
              <svg class="weather-icon" viewBox="0 0 128 128" width="144" height="144"
aria-hidden="true">
                <use :href="'/weather-icons.svg#' +
weatherIcon(weatherData.today.icon)"></use>
              </svg>
              <p>{{ weatherData.today.conditionDescription }}</p>
            </div>
            <p class="average-temperature">
              {{ formatTemperature(weatherData.today.averageTemperature) }}
            </p>
            <p class="average-label">Average temperature</p>
            <p
              class="temperature-category"
              :class="'category-' + weatherData.today.temperatureCategory.toLowerCase()"
            >
              {{ weatherData.today.temperatureCategory }}
            </p>
            <div class="high-low">
              <span>Low <strong>{{ formatTemperature(weatherData.today.minTemperature)
}}</strong></span>
              <span>High <strong>{{ formatTemperature(weatherData.today.maxTemperature)
}}</strong></span>
            </div>
          </section>
          <p v-else class="side-empty">
            Search a city to see its forecast and packing advice.
          </p>
        </aside>

        <main class="main-panel">
          <section v-if="loading" class="message-panel" aria-live="polite">
            <p>Looking up the forecast...</p>
          </section>

          <section v-if="error" class="message-panel error-panel" role="alert">
            <div>
              <h1>Couldn't load this forecast</h1>
              <p>{{ error }}</p>
            </div>
            <button class="retry-button" type="button" @click="searchWeather"
:disabled="loading">
              Try again
            </button>
          </section>

          <section v-if="!loading && !error && !weatherData" class="message-panel
empty-panel">
            <svg
              class="weather-icon empty-icon"
              viewBox="0 0 128 128"
              width="144"
              height="144"
              aria-hidden="true"
            >
              <use href="/weather-icons.svg#partly-cloudy-day"></use>
            </svg>
            <h1>Three-day forecast</h1>
            <p>Your temperature, wind, rainfall and air-quality details will appear
here.</p>
          </section>

          <div v-if="weatherData" class="weather-results">
            <section class="forecast-section" aria-labelledby="forecast-title">
              <div class="section-heading">
```

```
                <h1 id="forecast-title">Three-day forecast</h1>
                <p>Local forecast</p>
              </div>
              <p v-if="!weatherData.hasThreeDays" class="notice">
                OpenWeather returned fewer than three forecast days. The available days are
shown below.
              </p>
              <div class="table-scroll" tabindex="0" aria-label="Scrollable forecast
table">
                <table class="forecast-table">
                  <thead>
                    <tr>
                      <th scope="col">Day</th>
                      <th scope="col">Conditions</th>
                      <th scope="col">Temperature</th>
                      <th scope="col">Wind speed</th>
                      <th scope="col">Rainfall</th>
                    </tr>
                  </thead>
                  <tbody>
                    <tr v-for="day in weatherData.days" :key="day.date">
                      <th scope="row">{{ formatDate(day.date) }}</th>
                      <td class="weather-condition" data-label="Conditions">
                        <div class="condition-content">
                          <svg
                            class="weather-icon"
                            viewBox="0 0 128 128"
                            width="52"
                            height="52"
                            aria-hidden="true"
                          >
                            <use :href="'/weather-icons.svg#' +
weatherIcon(day.icon)"></use>
                          </svg>
                          <span>{{ day.conditionDescription }}</span>
                        </div>
                      </td>
                      <td data-label="Temperature">
                        <span class="temperature-range">
                          {{ formatTemperature(day.minTemperature) }} -
                          {{ formatTemperature(day.maxTemperature) }}
                        </span>
                        <span
                          class="table-category"
                          :class="'category-' + day.temperatureCategory.toLowerCase()"
                        >
                          {{ day.temperatureCategory }}
                        </span>
                      </td>
                      <td data-label="Wind speed">{{ formatNumber(day.averageWindSpeed) }}
m/s</td>
                      <td data-label="Rainfall">{{ formatRainfall(day.rainfall) }} mm</td>
                    </tr>
                  </tbody>
                </table>
              </div>
            </section>

            <div class="details-grid">
              <section class="packing-section" aria-labelledby="packing-title">
                <h2 id="packing-title">What to pack</h2>
                <p
                  class="packing-advice"
                  :class="weatherData.packing.umbrellaNeeded ? 'needs-umbrella' :
'no-umbrella'"
```

```
                >
                  {{ weatherData.packing.umbrellaNeeded ? "Bring an umbrella" : "No
umbrella needed" }}
                </p>
                <p class="temperature-advice">{{ weatherData.packing.temperatureAdvice
}}</p>
              </section>

              <section class="air-section" aria-labelledby="air-title">
                <div class="air-heading">
                  <h2 id="air-title">Air quality</h2>
                  <p
                    v-if="weatherData.airQuality.available"
                    class="air-status"
                    :class="'aqi-level-' + weatherData.airQuality.aqi"
                  >
                    <span class="status-dot" aria-hidden="true"></span>
                    {{ weatherData.airQuality.aqiLabel }} · AQI {{
weatherData.airQuality.aqi }}/5
                  </p>
                </div>

                <p v-if="!weatherData.airQuality.available" class="air-message">
                  {{ weatherData.airQuality.message }}
                </p>
                <p v-else-if="weatherData.airQuality.warnings.length === 0"
class="air-message">
                  All measured pollutants are within OpenWeather's Good range.
                </p>
                <ul v-else class="warning-list">
                  <li v-for="warning in weatherData.airQuality.warnings"
:key="warning.name">
                    <div class="warning-line">
                      <strong>{{ warning.name }}</strong>
                      <span>{{ formatNumber(warning.value) }} µg/m³ · {{ warning.category
}}</span>
                    </div>
                    <p>{{ warning.healthRisk }}</p>
                  </li>
                </ul>
              </section>
            </div>

            <section class="map-section" aria-labelledby="map-title">
              <div class="section-heading">
                <h2 id="map-title">Weather map</h2>
              </div>
              <p v-if="mapLoading" class="map-message" aria-live="polite">
                Loading map...
              </p>
              <p v-if="mapError" class="map-message map-error" role="status">
                {{ mapError }}
              </p>
              <div class="weather-map-frame">
                <div
                  id="weather-map"
                  class="weather-map"
                  :class="{ 'weather-map-hidden': pinLoading }"
                  :aria-hidden="pinLoading"
                  role="region"
                  :aria-label="'Weather map around ' + cityLabel"
                ></div>
                <div v-if="!pinLoading" class="map-layer-legend" role="group"
aria-label="Map layers">
                  <span class="map-layer-legend-title">Layers</span>
```

```
                  <div class="map-layer-controls">
                    <label class="map-layer-option">
                      <input
                        v-model="mapLayers.precipitation"
                        type="checkbox"
                        @change="setMapLayerVisibility('precipitation-layer',
mapLayers.precipitation)">
                      <span class="layer-swatch precipitation-swatch"
aria-hidden="true"></span>
                      <span>Precipitation</span>
                    </label>
                    <label class="map-layer-option">
                      <input
                        v-model="mapLayers.temperature"
                        type="checkbox"
                        @change="setMapLayerVisibility('temperature-layer',
mapLayers.temperature)">
                      <span class="layer-swatch temperature-swatch"
aria-hidden="true"></span>
                      <span>Temperature</span>
                    </label>
                  </div>
                </div>
                <div
                  v-if="weatherData && !pinLoading && (mapLayers.temperature ||
mapLayers.precipitation)"
                  class="map-weather-legend"
                  role="group"
                  aria-label="Weather at the selected map point"
                  aria-live="polite"
                >
                  <div v-if="mapLayers.temperature" class="map-current-reading">
                    <svg class="map-current-icon" viewBox="0 0 128 128" aria-hidden="true">
                      <use :href="'/weather-icons.svg#' +
weatherIcon(weatherData.currentWeather.icon)"></use>
                    </svg>
                    <div class="map-reading-copy">
                      <span class="map-reading-label">Current weather</span>
                      <strong class="map-current-temperature">
                        {{ formatTemperature(weatherData.currentWeather.temperature) }}
                      </strong>
                      <span class="map-current-condition">
                        {{ weatherData.currentWeather.conditionDescription }}
                      </span>
                    </div>
                  </div>
                  <div v-if="mapLayers.precipitation" class="map-rain-reading">
                    <span class="layer-swatch precipitation-swatch"
aria-hidden="true"></span>
                    <div class="map-reading-copy">
                      <span class="map-reading-label">Next likely rain</span>
                      <template v-if="weatherData.mapSummary.nextRain">
                        <strong class="map-rain-time">
                          {{ formatForecastTime(
                            weatherData.mapSummary.nextRain.time,
                            weatherData.mapSummary.timezoneOffset
                          ) }}
                        </strong>
                        <span class="map-rain-chance">
                          {{ weatherData.mapSummary.nextRain.chance }}% chance
                        </span>
                      </template>
                      <span v-else class="map-no-rain">
                        No likely rain in the next 3 days
                      </span>
```

```
                  </div>
                </div>
              </div>
              <div v-if="pinLoading" class="weather-map-placeholder" role="status"
aria-live="polite">
                Updating the forecast for this spot...
              </div>
            </div>
          </section>
        </div>
      </main>
    </div>
  </div>
</body>
</html>
```

public/style.css

```
/* colours and text */
:root {
  color-scheme: light;
  --page: #e2f2fa;
  --surface: #fff8e2;
  --main-surface: #e8f5fb;
  --tile: #ffffff;
  --ink: #1c303b;
  --muted: #435863;
  --line: #c9e2ec;
  --blue: #326b84;
  --sun-soft: #fff1c9;
  --amber: #745300;
  --red: #993e36;
  font-family: "Karla", "Trebuchet MS", sans-serif;
  color: var(--ink);
  background: var(--page);
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

* {
  box-sizing: border-box;
}

body {
  min-width: 320px;
  min-height: 100svh;
  margin: 0;
  background: var(--page);
  color: var(--ink);
  font-size: 18px;
  line-height: 1.5;
}

button,
input {
  font: inherit;
}

button {
  -webkit-tap-highlight-color: transparent;
}

.weather-shell {
  display: grid;
  grid-template-columns: minmax(290px, 350px) minmax(0, 1fr);
  width: 100%;
```

```
  min-height: 100svh;
  background: var(--main-surface);
}

/* search and today's weather */
.side-panel {
  display: flex;
  flex-direction: column;
  padding: 42px 34px;
  border-right: 1px solid var(--line);
  background: var(--surface);
}

.search-form {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  align-items: center;
  gap: 9px 12px;
  padding-bottom: 14px;
  border-bottom: 1px solid var(--line);
}

.search-form label {
  grid-column: 1 / -1;
  margin-bottom: -4px;
  font-size: 17px;
  font-weight: 600;
}

.search-form input {
  width: 100%;
  min-width: 0;
  min-height: 42px;
  padding: 8px 0;
  border: 0;
  outline: 0;
  background: transparent;
  color: var(--ink);
  font-size: 17px;
}

.search-form input::placeholder {
  color: var(--muted);
  opacity: 1;
}

.search-form input:focus-visible {
  outline: 2px solid var(--blue);
  outline-offset: 3px;
}

.search-form button,
.retry-button {
  min-height: 42px;
  padding: 9px 16px;
  border: 0;
  border-radius: 5px;
  background: var(--blue);
  color: white;
  cursor: pointer;
  font-size: 16px;
  font-weight: 600;
}

.search-form button:hover:not(:disabled),
```

```
.retry-button:hover:not(:disabled) {
  background: #105a84;
}

.search-form button:disabled,
.retry-button:disabled {
  cursor: not-allowed;
  opacity: 0.55;
}

.search-hint {
  grid-column: 1 / -1;
  margin: 0;
  color: var(--muted);
  font-size: 14px;
}

:focus-visible {
  outline: 2px solid var(--blue);
  outline-offset: 3px;
}

.today-summary {
  margin-top: 34px;
}

.city-name {
  margin: 0;
  font-size: 24px;
  font-weight: 600;
  line-height: 1.25;
  overflow-wrap: anywhere;
}

.day-label {
  margin: 5px 0 0;
  color: var(--muted);
  font-size: 17px;
}

.current-condition {
  margin: 24px auto 0;
  text-align: center;
}

.current-condition .weather-icon {
  display: block;
  width: 144px;
  height: 144px;
  object-fit: contain;
  margin: 0 auto;
}

.current-condition p {
  margin: -5px 0 0;
  color: var(--blue);
  font-size: 18px;
  font-weight: 600;
  text-transform: capitalize;
}

.average-temperature {
  margin: 18px 0 0 -5px;
  font-size: clamp(72px, 8vw, 96px);
  font-weight: 300;
```

```
  letter-spacing: -0.075em;
  line-height: 0.98;
  font-variant-numeric: tabular-nums;
}

.average-label {
  margin: 8px 0 0;
  color: var(--muted);
  font-size: 16px;
}

.temperature-category {
  margin: 18px 0 0;
  font-size: 19px;
  font-weight: 600;
}

.category-cold {
  color: var(--blue);
}

.category-mild {
  color: #4f6067;
}

.category-hot {
  color: var(--amber);
}

.high-low {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 14px;
  margin-top: 16px;
  padding-top: 14px;
  border-top: 1px solid var(--line);
  color: var(--muted);
  font-size: 16px;
}

.high-low span {
  display: flex;
  flex-direction: column;
  gap: 3px;
}

.high-low strong {
  color: var(--ink);
  font-size: 19px;
  font-weight: 600;
}

.side-empty {
  max-width: 25ch;
  margin: 30px 0 0;
  color: var(--muted);
  font-size: 17px;
}

/* forecast and info cards */
.main-panel {
  display: flex;
  align-items: center;
  min-width: 0;
  padding: 48px clamp(28px, 5vw, 72px);
```

```
  background: var(--main-surface);
}

.weather-results {
  width: 100%;
  max-width: 1000px;
  margin: 0 auto;
  align-self: flex-start;
}

.message-panel {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  justify-content: center;
  width: 100%;
  min-height: 390px;
}

.message-panel h1,
.forecast-section h1,
.air-section h2 {
  margin: 0;
  font-size: 29px;
  font-weight: 600;
  letter-spacing: -0.03em;
  line-height: 1.25;
}

.message-panel p {
  max-width: 42ch;
  margin: 9px 0 0;
  color: var(--muted);
  font-size: 18px;
}

.empty-panel .empty-icon {
  width: 152px;
  height: 152px;
  margin-bottom: 10px;
}

.error-panel {
  gap: 20px;
}

.error-panel h1 {
  color: var(--red);
}

.error-panel .retry-button {
  background: var(--red);
}

.section-heading {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 16px;
  margin-bottom: 20px;
}

.section-heading > p {
  margin: 0;
  color: var(--muted);
```

```
  font-size: 16px;
}

.notice {
  margin: 0 0 14px;
  padding: 10px 13px;
  border-radius: 8px;
  background: var(--sun-soft);
  color: var(--amber);
  font-size: 15px;
}

.table-scroll {
  overflow-x: auto;
}

.forecast-table {
  width: 100%;
  min-width: 800px;
  border-collapse: separate;
  border-spacing: 0 9px;
  text-align: left;
}

.forecast-table th,
.forecast-table td {
  padding: 13px 14px;
  vertical-align: middle;
}

.forecast-table thead th {
  padding-top: 0;
  padding-bottom: 4px;
  color: var(--muted);
  font-size: 15px;
  font-weight: 500;
}

.forecast-table tbody th,
.forecast-table tbody td {
  background: var(--tile);
  font-size: 16px;
}

.forecast-table tbody th {
  width: 25%;
  border-radius: 13px 0 0 13px;
  font-size: 17px;
  font-weight: 600;
}

.forecast-table tbody tr td:last-child {
  border-radius: 0 13px 13px 0;
}

.forecast-table td:nth-child(4),
.forecast-table td:nth-child(5),
.forecast-table thead th:nth-child(4),
.forecast-table thead th:nth-child(5) {
  text-align: right;
  white-space: nowrap;
  font-variant-numeric: tabular-nums;
}

.condition-content {
```

```
  display: flex;
  align-items: center;
  gap: 10px;
}

.condition-content .weather-icon {
  display: block;
  flex: 0 0 auto;
}

.condition-content span {
  text-transform: capitalize;
}

.temperature-range {
  display: block;
  color: var(--ink);
  font-size: 17px;
  font-weight: 600;
  white-space: nowrap;
  font-variant-numeric: tabular-nums;
}

.table-category {
  display: block;
  margin-top: 3px;
  font-size: 15px;
  font-weight: 500;
}

.details-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 18px;
  margin-top: 28px;
}

.map-section {
  margin-top: 32px;
}

.map-section .section-heading {
  margin-bottom: 5px;
}

.map-section h2 {
  margin: 0;
  font-size: 24px;
  font-weight: 600;
}

.map-message {
  margin: 0 0 10px;
  color: var(--muted);
  font-size: 15px;
}

.map-layer-controls {
  display: grid;
  gap: 7px;
}

.map-layer-legend {
  position: absolute;
  top: 12px;
```

```
  left: 12px;
  z-index: 3;
  display: grid;
  gap: 8px;
  padding: 10px 12px;
  border-radius: 14px;
  background: rgba(255, 255, 255, 0.97);
  box-shadow: 0 2px 10px rgba(22, 48, 58, 0.16);
}

.map-layer-legend-title {
  color: var(--muted);
  font-size: 12px;
  font-weight: 600;
}

.map-layer-option {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  font-size: 13px;
  font-weight: 600;
  cursor: pointer;
}

.layer-swatch {
  width: 12px;
  height: 12px;
  border-radius: 50%;
}

.precipitation-swatch {
  background: #258ac4;
}

.temperature-swatch {
  background: linear-gradient(135deg, #39a7d5 0 35%, #f0bd45 36% 68%, #e66d4c 69%);
}

.map-layer-option input {
  width: 16px;
  height: 16px;
  margin: 0;
  accent-color: var(--blue);
}

.map-error {
  color: var(--red);
}

.weather-map-frame {
  position: relative;
  height: 400px;
  overflow: hidden;
  border: 1px solid var(--line);
  border-radius: 12px;
  background: #dceaf0;
}

.weather-map {
  width: 100%;
  height: 100%;
  overflow: hidden;
  background: transparent;
}
```

```
.weather-map-frame .mapboxgl-ctrl-attrib,
.weather-map-frame .mapboxgl-ctrl-attrib-inner {
  font-size: 9px !important;
  line-height: 1.2 !important;
}

.weather-map-frame .mapboxgl-ctrl-attrib {
  padding: 2px 4px;
}

.map-weather-legend {
  position: absolute;
  right: 12px;
  bottom: 28px;
  z-index: 3;
  display: grid;
  gap: 11px;
  width: min(250px, calc(100% - 24px));
  padding: 14px;
  border-radius: 16px;
  background: rgba(255, 255, 255, 0.97);
  box-shadow: 0 2px 12px rgba(22, 48, 58, 0.18);
}

.map-current-reading {
  display: flex;
  align-items: center;
  gap: 10px;
}

.map-current-icon {
  width: 42px;
  height: 42px;
  flex: 0 0 42px;
}

.map-reading-copy {
  display: grid;
  gap: 2px;
  min-width: 0;
}

.map-reading-label {
  color: var(--muted);
  font-size: 12px;
  font-weight: 600;
}

.map-current-temperature {
  color: var(--ink);
  font-size: 23px;
  font-variant-numeric: tabular-nums;
  line-height: 1.1;
}

.map-current-condition,
.map-rain-chance {
  color: var(--muted);
  font-size: 13px;
  text-transform: capitalize;
}

.map-rain-reading {
  display: grid;
```

```
  grid-template-columns: 12px minmax(0, 1fr);
  align-items: start;
  gap: 10px;
  padding-top: 10px;
  border-top: 1px solid var(--line);
}

.map-rain-reading .layer-swatch {
  margin-top: 4px;
}

.map-rain-time,
.map-no-rain {
  color: var(--ink);
  font-size: 15px;
  font-weight: 600;
  line-height: 1.25;
}

.weather-map-hidden {
  visibility: hidden;
}

.weather-map-placeholder {
  position: absolute;
  inset: 0;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 10px;
  padding: 20px;
  background: #dceaf0;
  color: var(--ink);
  font-size: 17px;
  font-weight: 600;
  text-align: center;
}

.weather-map-placeholder::before {
  width: 16px;
  height: 16px;
  flex: 0 0 16px;
  border: 2px solid var(--line);
  border-top-color: var(--blue);
  border-radius: 50%;
  animation: map-loading-spin 0.8s linear infinite;
  content: "";
}

@keyframes map-loading-spin {
  to { transform: rotate(360deg); }
}

@media (prefers-reduced-motion: reduce) {
  .weather-map-placeholder::before {
    animation: none;
  }
}

/* stack the cards on smaller screens */
@media (max-width: 1200px) {
  .details-grid {
    grid-template-columns: 1fr;
  }
```

```
}

.packing-section,
.air-section {
  min-width: 0;
  padding: 22px 24px;
  border-radius: 13px;
  background: #fff;
}
.packing-section h2,
.air-section h2 {
  margin: 0;
  font-size: 24px;
  font-weight: 600;
}

.packing-advice {
  margin: 18px 0 0;
  font-size: 18px;
  font-weight: 600;
}

.needs-umbrella {
  color: #365b6b;
}

.no-umbrella {
  color: #336e4d;
}

.temperature-advice {
  margin: 9px 0 0;
  color: var(--ink);
  font-size: 16px;
}

.air-heading {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 18px;
}

.air-status {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  margin: 0;
  color: var(--aqi-color);
  font-size: 16px;
  font-weight: 600;
}

.aqi-level-1 {
  --aqi-color: #326344;
}

.aqi-level-2 {
  --aqi-color: #755900;
}

.aqi-level-3 {
  --aqi-color: #814817;
}
```

```
.aqi-level-4 {
  --aqi-color: #872f2a;
}

.aqi-level-5 {
  --aqi-color: #692b3c;
}

.status-dot {
  width: 10px;
  height: 10px;
  flex: 0 0 auto;
  border-radius: 50%;
  background: currentColor;
}

.air-message {
  margin: 0;
  color: var(--ink);
  font-size: 16px;
}

.warning-list {
  margin: 0;
  padding: 0;
  list-style: none;
}

.warning-list li + li {
  margin-top: 16px;
}

.warning-line {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 12px;
  font-size: 16px;
}

.warning-line strong {
  font-weight: 600;
}

.warning-line span {
  color: var(--ink);
  font-size: 15px;
  text-align: right;
}

.warning-list li p {
  margin: 5px 0 0;
  color: var(--muted);
  font-size: 15px;
}

/* move the sidebar above the forecast */
@media (max-width: 1000px) {
  .weather-shell {
    display: block;
  }

  .side-panel {
    padding: 30px 28px;
```

```
    border-right: 0;
    border-bottom: 1px solid var(--line);
  }

  .today-summary {
    max-width: 460px;
  }

  .current-condition {
    margin-left: 0;
  }

  .main-panel {
    padding: 36px 28px;
  }
}

/* phone layout */
@media (max-width: 650px) {
  .side-panel {
    padding: 24px 20px 28px;
  }

  .today-summary {
    margin-top: 26px;
  }

  .current-condition {
    margin-top: 16px;
  }

  .current-condition .weather-icon {
    width: 124px;
    height: 124px;
  }

  .average-temperature {
    margin-top: 12px;
    font-size: 76px;
  }

  .main-panel {
    min-height: 380px;
    padding: 30px 20px 36px;
  }

  .message-panel {
    min-height: 280px;
  }

  .details-grid {
    gap: 16px;
    margin-top: 24px;
  }

  .forecast-table {
    min-width: 860px;
  }

  .section-heading {
    align-items: flex-start;
  }

  .section-heading > p {
    text-align: right;
```

```
  }

  .air-heading {
    flex-direction: column;
    align-items: flex-start;
    gap: 7px;
  }

  .warning-line {
    align-items: flex-start;
    flex-direction: column;
    gap: 2px;
  }

  .warning-line span {
    text-align: left;
  }

  .map-section {
    margin-top: 26px;
  }
  .weather-map {
    height: 100%;
  }

  .weather-map-frame {
    height: 300px;
  }
}
```

public/weather-icons.svg

```
<svg xmlns="http://www.w3.org/2000/svg">
  <symbol id="sun" viewBox="0 0 128 128">
    <g fill="none" stroke="#D8B247" stroke-linecap="round" stroke-width="6">
      <path d="M64 10v14M64 104v14M10 64h14M104 64h14M26 26l10 10M92 92l10 10M102 26 92
36M36 92 26 102" />
    </g>
    <circle cx="64" cy="64" r="32" fill="#F2CF68" />
  </symbol>

  <symbol id="moon" viewBox="0 0 128 128">
    <path d="M91 19a49 49 0 1 0 18 89A51 51 0 0 1 91 19Z" fill="#E7C96F" />
  </symbol>

  <symbol id="partly-cloudy-day" viewBox="0 0 128 128">
    <g fill="none" stroke="#D8B247" stroke-linecap="round" stroke-width="5">
      <path d="M77 12v10M108 44H98M99 21l-7 7M54 21l7 7" />
    </g>
    <circle cx="77" cy="44" r="25" fill="#F2CF68" />
    <path d="M31 105h65a21 21 0 0 0 0-42 31 31 0 0 0-60-8 25 25 0 0 0-5 50Z" fill="#A3BEC7"
/>
  </symbol>

  <symbol id="partly-cloudy-night" viewBox="0 0 128 128">
    <path d="M89 17a28 28 0 0 0 26 42A38 38 0 1 1 89 17Z" fill="#E7C96F" />
    <path d="M31 105h65a21 21 0 0 0 0-42 31 31 0 0 0-60-8 25 25 0 0 0-5 50Z" fill="#A3BEC7"
/>
  </symbol>

  <symbol id="cloud" viewBox="0 0 128 128">
    <path d="M29 101h68a22 22 0 0 0 0-44 32 32 0 0 0-61-8 26 26 0 0 0-7 52Z" fill="#A3BEC7"
/>
  </symbol>

  <symbol id="rain" viewBox="0 0 128 128">
```

```
    <path d="M29 84h68a22 22 0 0 0 0-44 32 32 0 0 0-61-8 26 26 0 0 0-7 52Z" fill="#A3BEC7"
/>
    <g fill="#6D9FB2">
      <path d="M42 92c0 8-10 8-10 0 0-5 5-11 5-11s5 6 5 11ZM70 102c0 8-10 8-10 0 0-5 5-11
5-11s5 6 5 11ZM98 92c0 8-10 8-10 0 0-5 5-11 5-11s5 6 5 11Z" />
    </g>
  </symbol>

  <symbol id="storm" viewBox="0 0 128 128">
    <path d="M29 80h68a22 22 0 0 0 0-44 32 32 0 0 0-61-8 26 26 0 0 0-7 52Z" fill="#A3BEC7"
/>
    <path d="m66 70-18 28h16l-5 22 25-34H67l9-16Z" fill="#E7C96F" />
  </symbol>
  <symbol id="snow" viewBox="0 0 128 128">
    <path d="M29 80h68a22 22 0 0 0 0-44 32 32 0 0 0-61-8 26 26 0 0 0-7 52Z" fill="#A3BEC7"
/>
    <g fill="#6D9FB2">
      <circle cx="39" cy="98" r="5" />
      <circle cx="65" cy="108" r="5" />
      <circle cx="91" cy="98" r="5" />
    </g>
  </symbol>

  <symbol id="mist" viewBox="0 0 128 128">
    <path d="M29 77h68a22 22 0 0 0 0-44 32 32 0 0 0-61-8 26 26 0 0 0-7 52Z" fill="#A3BEC7"
/>
    <g fill="none" stroke="#6D9FB2" stroke-linecap="round" stroke-width="6">
      <path d="M25 94h78M38 109h52" />
    </g>
  </symbol>
</svg>
```

public/openapi.json

```
{
  "openapi": "3.0.3",
  "info": {
    "title": "Weather Planner API",
    "version": "1.0.0",
    "description": "Weather search and map endpoints used by the Weather Planner."
  },
  "tags": [
    {
      "name": "Weather",
      "description": "Weather lookups by city or map coordinates."
    },
    {
      "name": "Map",
      "description": "Map configuration and proxied weather tiles."
    }
  ],
  "paths": {
    "/api/config": {
      "get": {
        "tags": ["Map"],
        "summary": "Get public map configuration",
        "description": "Returns the public Mapbox token used by the browser to load the
basemap. An empty token means Mapbox is not configured.",
        "responses": {
          "200": {
            "description": "Mapbox configuration for the browser.",
            "content": {
              "application/json": {
                "schema": {
                  "type": "object",
```

```
                  "required": ["mapboxToken"],
                  "properties": {
                    "mapboxToken": {
                      "type": "string",
                      "description": "Public Mapbox access token, or an empty string if not
configured."
                    }
                  }
                }
              }
            }
          }
        }
      },
    "/api/weather": {
      "get": {
        "tags": ["Weather"],
        "summary": "Get the forecast for one city",
        "description": "Looks up the city with OpenWeather Geocoding, then returns its
current weather, forecast, packing advice and air quality.",
        "parameters": [
          {
            "name": "city",
            "in": "query",
            "required": true,
            "description": "City name, optionally followed by a country code.",
            "schema": {
              "type": "string",
              "example": "Dublin, IE"
            }
          }
        ],
        "responses": {
          "200": {
            "description": "Current weather, forecast, packing advice and air quality for
the city.",
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/WeatherResult"
                }
              }
            }
          },
          "400": {
            "$ref": "#/components/responses/BadRequest"
          },
          "404": {
            "$ref": "#/components/responses/CityNotFound"
          },
          "500": {
            "$ref": "#/components/responses/MissingApiKey"
          },
          "502": {
            "$ref": "#/components/responses/WeatherServiceError"
          },
          "default": {
            "$ref": "#/components/responses/OtherError"
          }
        }
      }
    },
    "/api/weather-at": {
      "get": {
```

```
        "tags": ["Weather"],
        "summary": "Get the forecast for a map point",
        "description": "Uses the clicked coordinates for current weather, forecast and
air-quality lookups, and reverse geocoding for the nearby place name.",
        "parameters": [
          {
            "name": "lat",
            "in": "query",
            "required": true,
            "schema": {
              "type": "number",
              "minimum": -90,
              "maximum": 90,
              "example": 53.3498
            }
          },
          {
            "name": "lon",
            "in": "query",
            "required": true,
            "schema": {
              "type": "number",
              "minimum": -180,
              "maximum": 180,
              "example": -6.2603
            }
          }
        ],
        "responses": {
          "200": {
            "description": "Current weather, forecast, packing advice and air quality for
the selected point.",
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/WeatherResult"
                }
              }
            }
          },
          "400": {
            "$ref": "#/components/responses/BadRequest"
          },
          "500": {
            "$ref": "#/components/responses/MissingApiKey"
          },
          "502": {
            "$ref": "#/components/responses/WeatherServiceError"
          },
          "default": {
            "$ref": "#/components/responses/OtherError"
          }
        }
      }
    },
    "/api/weather-map/{layer}/{z}/{x}/{y}.png": {
      "get": {
        "tags": ["Map"],
        "summary": "Get a weather map tile",
        "description": "Express fetches a temperature or precipitation tile from
OpenWeather so the OpenWeather API key stays on the server.",
        "parameters": [
          {
            "name": "layer",
            "in": "path",
```

```
          "required": true,
          "schema": {
            "type": "string",
            "enum": ["precipitation_new", "temp_new"]
          }
        },
        {
          "name": "z",
          "in": "path",
          "required": true,
          "schema": {
            "type": "integer",
            "minimum": 0,
            "maximum": 18
          }
        },
        {
          "name": "x",
          "in": "path",
          "required": true,
          "description": "Tile column. Must be less than 2 to the power of z.",
          "schema": {
            "type": "integer",
            "minimum": 0
          }
        },
        {
          "name": "y",
          "in": "path",
          "required": true,
          "description": "Tile row. Must be less than 2 to the power of z.",
          "schema": {
            "type": "integer",
            "minimum": 0
          }
        }
      ],
      "responses": {
        "200": {
          "description": "A PNG tile for the requested weather layer.",
          "content": {
            "image/png": {
              "schema": {
                "type": "string",
                "format": "binary"
              }
            }
          }
        },
        "400": {
          "description": "The layer name or tile coordinates are invalid."
        },
        "500": {
          "description": "The server does not have an OpenWeather API key."
        },
        "502": {
          "description": "The server could not fetch the tile from OpenWeather."
        },
        "default": {
          "description": "OpenWeather returned an error status for this tile."
        }
      }
    }
  }
},
```

```
"components": {
  "schemas": {
    "ApiError": {
      "type": "object",
      "properties": {
        "error": {
          "type": "string"
        }
      }
    },
    "City": {
      "type": "object",
      "properties": {
        "name": {
          "type": "string"
        },
        "state": {
          "type": "string"
        },
        "country": {
          "type": "string"
        }
      }
    },
    "CurrentWeather": {
      "type": "object",
      "properties": {
        "temperature": {
          "type": "number"
        },
        "conditionDescription": {
          "type": "string"
        },
        "icon": {
          "type": "string"
        }
      }
    },
    "DaySummary": {
      "type": "object",
      "properties": {
        "date": {
          "type": "string",
          "format": "date"
        },
        "icon": {
          "type": "string"
        },
        "conditionDescription": {
          "type": "string"
        },
        "minTemperature": {
          "type": "number"
        },
        "maxTemperature": {
          "type": "number"
        },
        "averageTemperature": {
          "type": "number"
        },
        "temperatureCategory": {
          "type": "string",
          "enum": [
            "Cold",
            "Mild",
```

```
            "Hot"
          ]
        },
        "averageWindSpeed": {
          "type": "number"
        },
        "rainfall": {
          "type": "number"
        },
        "rainExpected": {
          "type": "boolean"
        }
      }
    },
    "PackingAdvice": {
      "type": "object",
      "properties": {
        "umbrellaNeeded": {
          "type": "boolean"
        },
        "umbrellaAdvice": {
          "type": "string"
        },
        "temperatureAdvice": {
          "type": "string"
        }
      }
    },
    "PollutantWarning": {
      "type": "object",
      "properties": {
        "name": {
          "type": "string"
        },
        "value": {
          "type": "number"
        },
        "category": {
          "type": "string"
        },
        "healthRisk": {
          "type": "string"
        }
      }
    },
    "AirQuality": {
      "type": "object",
      "required": [
        "available"
      ],
      "properties": {
        "available": {
          "type": "boolean"
        },
        "message": {
          "type": "string"
        },
        "aqi": {
          "type": "integer",
          "nullable": true
        },
        "aqiLabel": {
          "type": "string"
        },
        "warnings": {
```

```
          "type": "array",
          "items": {
            "$ref": "#/components/schemas/PollutantWarning"
          }
        }
      }
    },
    "WeatherResult": {
      "type": "object",
      "properties": {
        "city": {
          "$ref": "#/components/schemas/City"
        },
        "currentWeather": {
          "$ref": "#/components/schemas/CurrentWeather"
        },
        "today": {
          "$ref": "#/components/schemas/DaySummary"
        },
        "days": {
          "type": "array",
          "items": {
            "$ref": "#/components/schemas/DaySummary"
          }
        },
        "packing": {
          "$ref": "#/components/schemas/PackingAdvice"
        },
        "airQuality": {
          "$ref": "#/components/schemas/AirQuality"
        },
        "hasThreeDays": {
          "type": "boolean"
        },
        "coordinates": {
          "type": "object",
          "properties": {
            "lat": {
              "type": "number"
            },
            "lon": {
              "type": "number"
            }
          }
        },
        "mapSummary": {
          "type": "object",
          "properties": {
            "timezoneOffset": {
              "type": "integer"
            },
            "nextRain": {
              "type": "object",
              "nullable": true,
              "properties": {
                "time": {
                  "type": "integer"
                },
                "chance": {
                  "type": "integer"
                }
              }
            }
          }
        }
```

```
        }
      }
    },
    "responses": {
      "BadRequest": {
        "description": "A required query parameter is missing or invalid.",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/ApiError"
            }
          }
        }
      },
      "CityNotFound": {
        "description": "OpenWeather could not find the requested city.",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/ApiError"
            }
          }
        }
      },
      "MissingApiKey": {
        "description": "The server does not have an OpenWeather API key.",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/ApiError"
            }
          }
        }
      },
      "WeatherServiceError": {
        "description": "The server could not connect to OpenWeather or did not receive
enough forecast data.",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/ApiError"
            }
          }
        }
      },
      "OtherError": {
        "description": "Another error returned by OpenWeather.",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/ApiError"
            }
          }
        }
      }
    }
  }
}
```

public/api-docs.html

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Weather Planner API</title>
    <link rel="stylesheet" href="https://unpkg.com/swagger-ui-dist@5.32.0/swagger-ui.css">
</head>
<body>
    <div id="swagger-ui"></div>

    <script src="https://unpkg.com/swagger-ui-dist@5.32.0/swagger-ui-bundle.js"></script>
    <script>
        window.onload = () => {
            SwaggerUIBundle({
                url: "/openapi.json",
                dom_id: "#swagger-ui"
            });
        };
    </script>
</body>
</html>
```

Appendix B: Academic Integrity Declaration

I understand that this is an individual assessment and that collaboration is not permitted. I have not received any assistance with my work for this assessment. Where I have used the published work of others, I have indicated this with appropriate citation.

I have not and will not share any part of my work on this assessment, directly or indirectly, with any other student. I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at http://www.tcd.ie/calendar.

I understand that by returning this declaration with my work, I am agreeing with the above statement.

Name: Liliana Berenguer Jin
Date: 21st October 2026